

SQL, Power BI, Python

le bon ordre d'apprentissage

Repère 2026 • guide en 8 étapes



Pour chaque étape : la compétence visée, le livrable qui la prouve, l'erreur classique du débutant, et le test qui autorise à passer à la suite.

Sur le terrain, la plupart des reconversions apprennent les outils dans le désordre : elles ouvrent Power BI avant de savoir écrire une jointure, ou elles codent en Python avant de savoir filtrer une table. Résultat : elles utilisent une fraction des fonctionnalités et n'arrivent pas à défendre leurs chiffres en entretien. Ce guide fixe l'ordre qui construit une vraie compétence, étape par étape, avec la preuve à produire à chaque palier.

Fondations SQL

Ici, tu construis le socle que tout le reste utilisera : lire, filtrer, joindre et modéliser une donnée avant de la montrer.

1. SQL, lire et filtrer des données

Compétence : Tu apprends à interroger une table brute sans passer par un outil graphique : choisir les bonnes colonnes, restreindre les lignes utiles avec `WHERE`, ordonner un résultat lisible.

```
SELECT client_id, montant, date_commande
FROM commandes
WHERE statut = 'payee'
ORDER BY date_commande DESC;
```

Le test qui autorise à passer

Tu dois écrire cette requête seul, sur un jeu de données que tu n'as jamais vu, sans regarder de correction.

Livrable

Une requête filtrée et triée qui tourne du premier coup sur une table inconnue.

Erreur classique

Copier une syntaxe vue en tuto sans comprendre pourquoi `WHERE` précède `ORDER BY` dans la logique du résultat.

Test de passage

Isoler, sans aide, les commandes payées d'un client sur une période donnée, triées de la plus récente à la plus ancienne.

2. SQL, agréger et joindre plusieurs tables

Compétence : Tu apprends à combiner une table de clients et une table de commandes, puis à résumer avec `GROUP BY`. C'est ici qu'on distingue un débutant qui sait écrire une requête d'un junior qui la fait tenir face à plusieurs tables.

```
SELECT c.id, c.nom, COUNT(o.id) AS nb_commandes,
       SUM(CASE WHEN o.statut = 'payee' THEN o.montant ELSE 0 END) AS total_paye
FROM clients c
LEFT JOIN commandes o ON o.client_id = c.id
GROUP BY c.id, c.nom;
```

Pourquoi ce join précis

Un `INNER JOIN` ici ferait disparaître les clients sans commande. Le `LEFT JOIN` les garde à zéro, ce qui change complètement un taux d'inactivité calculé dessus.

Livrable

Une agrégation par client qui inclut les clients à zéro commande, vérifiée ligne par ligne contre la table source.

Erreur classique

Confondre `INNER JOIN` et `LEFT JOIN` sur des données réelles : le compte de clients change sans message d'erreur visible.

Test de passage

Refaire le calcul avec `INNER JOIN` puis `LEFT JOIN` et expliquer pourquoi les deux totaux diffèrent.

3. Modéliser un schéma de données lisible

Compétence : Tu apprends à séparer une table de faits, une ligne par commande, des tables de dimensions comme client, produit ou date. C'est le pont qu'on saute trop souvent avant d'ouvrir Power BI.

Ce que ce pont évite

Un schéma non modélisé oblige Power BI à recalculer des jointures lourdes à chaque clic, et peut dupliquer des lignes sans erreur visible.

Livrable

Un schéma en étoile documenté : une table de faits commandes, des dimensions clients, produits et dates reliées par des clés uniques.

Erreur classique

Importer les tables brutes telles quelles et laisser Power BI deviner les relations tout seul.

Test de passage

Nommer sans hésiter la table de faits et les 3 dimensions d'un jeu de données, et vérifier qu'aucune clé n'a de doublon.

À retenir, palier SQL

Les trois tests de passage ne portent pas sur la syntaxe : ils vérifient ta capacité à défendre un chiffre, filtrer juste, joindre sans perdre de lignes, modéliser avant de visualiser.

Étape 1

Filtrer et trier seul, sans correction sous les yeux.

Étape 2

Justifier le choix du join, pas seulement l'écrire.

Étape 3

Nommer faits et dimensions avant d'ouvrir Power BI.

Visualisation Power BI

Ici, Power BI arrive après la donnée modélisée, jamais en premier réflexe.

4. Power BI, construire un rapport qu'on peut relire

Compétence : Tu apprends à charger le schéma modélisé à l'étape 3, à construire les relations dans le modèle, puis à poser les premiers visuels sans empiler les mesures au hasard.

Le signal que ça n'a pas été fait dans l'ordre

Un rapport construit avant la modélisation doit être refait dès le premier changement de source : chaque nouvelle colonne casse une relation devinée par l'outil.

Livrable

Un rapport avec relations explicites entre faits et dimensions, visibles dans la vue modèle, sans relation en pointillé non expliquée.

Erreur classique

Empiler les visuels avant d'avoir vérifié le sens des relations, en laissant Power BI deviner les clés.

Test de passage

Changer une source de données et vérifier que le rapport se recalcule sans erreur de relation.

5. Power BI, écrire une mesure DAX simple

Compétence : Tu apprends à écrire une mesure avec `CALCULATE` qui répond à une question précise, plutôt qu'une colonne calculée figée. C'est le seuil qui rend un rapport interrogeable au lieu de statique.

```
Ventes payees =
CALCULATE(
    SUM(commandes[montant]),
    commandes[statut] = "payee"
)
```

Pourquoi une mesure et pas une colonne

Une colonne calculée se fige à l'actualisation. Une mesure se recalcule selon le filtre du visuel : change le segment client, et le total suit.

Livable

Une mesure qui recalcule un total conditionnel selon le filtre actif du visuel, testée avec deux segments différents.

Erreur classique

Créer une colonne calculée pour un total qui devrait réagir aux filtres du rapport.

Test de passage

Appliquer un segment client sur le visuel et vérifier que la mesure change de valeur sans toucher la formule.

❗ Ce que Power BI ne remplace pas de SQL

Le palier suivant présume que la donnée qui arrive dans Power BI est déjà propre. Ce n'est vrai que si SQL a fait le tri en amont.

Ce que Power BI fait bien

Rendre lisible un modèle déjà propre, calculer des mesures qui réagissent aux filtres, partager un visuel interactif.

Ce que Power BI ne corrige pas

Une jointure qui a dupliqué des lignes en amont, un filtre SQL oublié, une clé mal typée : le visuel affiche le chiffre faux avec la même confiance qu'un chiffre juste.

Python et portfolio

Ici, Python rend reproductible ce que tu sais déjà faire à la main, puis prouve l'ordre entier dans un seul projet.

6. Python, manipuler des données avec pandas

Compétence : Tu apprends à reproduire en pandas un nettoyage déjà fait une fois à la main ou en SQL : charger, filtrer, regrouper. Python arrive après la visualisation, comme outil de reproductibilité, pas de découverte.

```
import pandas as pd

df = pd.read_csv("commandes.csv")
df_payees = df[df["statut"] == "payee"]
resume = df_payees.groupby("client_id")["montant"].sum()
```

Pourquoi ce n'est pas la première étape

Sans les réflexes SQL et le sens de la donnée acquis en amont, pandas devient un éditeur de tableau lent, pas un outil d'automatisation.

Livable

Un notebook qui reproduit, en quelques lignes, la colonne total_paye déjà vérifiée à la main en SQL, client par client.

Erreur classique

Utiliser pandas pour explorer une donnée jamais interrogée en SQL, au lieu de vérifier un résultat déjà connu.

Test de passage

Comparer, pour chaque client, le total pandas et la colonne total_paye de la requête SQL de l'étape 2 : ils doivent être strictement identiques.

7. Python, automatiser un nettoyage reproductible

Compétence : Tu apprends à transformer un script écrit une fois en fonction paramétrée, réutilisable sur un nouveau fichier sans tout réécrire.

```
def nettoyer_commandes(chemin_csv, statut="payee"):
    df = pd.read_csv(chemin_csv)
    df = df[df["statut"] == statut]
    df["date_commande"] = pd.to_datetime(df["date_commande"])
    return df
```

Le signal qui distingue un script d'un outil

Un script ponctuel marche une fois, sur un fichier précis. Une fonction paramétrée marche sur le prochain fichier sans modification du code.

Livrable

Une fonction avec au moins un paramètre, testée sur deux fichiers différents sans modifier son code.

Erreur classique

Copier-coller le script précédent et changer le nom du fichier en dur à chaque nouvelle donnée.

Test de passage

Faire tourner la fonction sur un fichier jamais vu et obtenir un résultat propre du premier coup.

8. Portfolio, un projet qui combine les trois outils

Compétence : Tu apprends à enchaîner les trois paliers sur un seul jeu de données : SQL pour interroger et modéliser, Power BI pour le rapport, Python pour automatiser le nettoyage en amont. C'est le livrable qui prouve l'ordre entier.

Ce qu'un recruteur vérifie en premier

Pas le nombre de lignes de code, mais si les trois outils s'enchaînent dans un ordre qui a un sens : la donnée propre avant le visuel, le visuel avant l'automatisation.

Livrable

Un dépôt avec un script Python de nettoyage, un schéma documenté et un rapport Power BI construit dessus, plus un README qui explique l'ordre suivi.

Erreur classique

Présenter trois projets isolés, un projet SQL, un projet Power BI, un projet Python, sans lien entre eux.

Test de passage

Expliquer en entretien, sans notes, pourquoi tu as modélisé la donnée avant d'ouvrir Power BI, et montrer la ligne du README qui le prouve.

À retenir, l'ordre compte plus que la liste d'outils

On me demande souvent quel outil apprendre en premier. Ma réponse ne change pas : ce n'est jamais une question d'outil, c'est une question d'ordre. Un junior qui sait écrire une jointure SQL propre progresse plus vite sur Power BI qu'un junior qui a suivi dix tutos DAX sans savoir filtrer une table. Retiens le test de passage de chaque étape, pas la liste des logiciels.

Si tu ne devais garder qu'une phrase de ce guide : ne change pas d'outil tant que tu n'as pas le livrable qui prouve l'étape précédente.

Fait main, en solo.

Si ce guide t'a aidé à voir l'ordre plus clair, partage-le autour de toi : il peut débloquent une reconversion bloquée sur le mauvais outil. Et si tu veux que je détaille une étape précise, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

